

## Error Description

### Error:Stale Cache Problem

The application used Redis caching to store project, task, and subtask data for 600 seconds. When a resource was fetched, the result was cached. However, if a new resource was created or deleted before the cache expired, subsequent requests would serve outdated information. This led to situations where, for example, deleted projects would still appear to exist, or new projects would not be found until cache expiry.

## Solution

### Cache Expiration and Invalidation

The cache expiration was reduced from 600 seconds to 120 seconds to minimize stale data issues. Additionally, cache keys are now explicitly deleted from Redis whenever related resources are created, updated, or deleted. For example, when a project is deleted, its cache entry is also removed.

Python

```
# When deleting a project
```

```
cache_delete(f"project_id:{project_id}")
```

```
db.delete(project)
```

```
db.commit()
```

### ErrorSchema Access Issue for Super Admin

The database uses a multi-tenant setup.

- The public schema contains tables for organizations, roles, and users.
- Each organization has a separate schema (e.g., org\_2) for projects, tasks, and subtasks.

Only organization admins (org\_admin) were correctly accessing their organization's schema using the get\_db(org\_id) function. Super admins, whose org\_id is None, defaulted to the public schema, which does not contain project, task, or subtask tables. As a result, a super admin trying to access projects would encounter errors such as "Organization schema org\_None not found."

### Super Admin Cross-Schema Access

Super admins need the ability to view all projects across all organizations. This was solved by fetching the list of all organizations from the public schema, then iterating through each organization's schema to collect and aggregate project data.

Python

```
if user.role_name.lower() == "super_admin":
    all_projects = []
    orgs = db.query(Organization).all()
    for org in orgs:
        try:
            with next(get_tenant_db(org.id)) as tenant_db:
                projects = tenant_db.query(Project).all()
                all_projects.extend(projects)
        except Exception as e:
            logger.error(f"Error fetching projects for org {org.id}: {e}")
            raise HTTPException(status_code=500, detail="Internal server error")
    projects = all_projects
else:
    if not user.org_id:
        raise HTTPException(status_code=400, detail="User not in any organization")
    projects = get_projects_by_org_id(db, user.org_id, user)
```